

Fused Attack Graphs for Budgeted Remediation: An Open, Measured Evaluation of LINCHPIN

Rakshit
VIT Chennai

Preprint, October 2026 (not peer reviewed)

Abstract

Vulnerability management usually ranks findings one at a time, by CVSS, EPSS or membership in CISA’s Known Exploited Vulnerabilities (KEV) catalog. Choosing a minimum set of fixes that disconnects an attack graph is a classical alternative, but open, reproducible evidence about *which data* such plans need is scarce. We present LINCHPIN, an open-source, read-only tool that fuses already-collected scanner exports, Active Directory identity data and network segmentation into one attack graph with exploit-intelligence edge costs, and recommends a budgeted, evidence-carrying set of fixes: the exact minimum vertex cut when it fits the budget, a greedy set cover otherwise. On 200 seeded topologies in four families with real CVE parameters, adding identity data to the scanner graph lifts the three-fix disconnect rate from 37% [29, 45] to 100% [97.5, 100] of the 150 topologies where a three-fix cut exists (segmentation and exploit-intelligence costs change the number of fixes needed and the attacker-cost gain, not this rate), against 3% [1, 7] for a KEV-then-EPSS patch queue (paired exact McNemar $p < 10^{-4}$). The exact budgeted interdiction program of Israeli and Wood reaches the same rate and a greedy interdiction heuristic 97%; where no cut fits the budget, LINCHPIN attains 0.83 [0.77, 0.88] of the optimal rise in the attacker’s cheapest-path cost. In a lab of five deliberately outdated container images scanned inside continuous integration, one upgrade disconnects the database (as do betweenness and both interdiction baselines) while a CVSS-first queue needs three. We also reproduce part of a published EPSS-versus-CVSS comparison with public labels. Code, data pointers and every result are public.

1 Introduction

A remediation queue sorted by CVSS treats a critical vulnerability on an unreachable host as urgent and two moderate ones that chain to the domain controller as routine. Exploit prediction (EPSS [10]) and the KEV catalog sharpen the per-finding signal, but they still score findings in isolation. Attack graphs [15, 16, 14] model how an attacker chains footholds, and minimum-cost hardening [13, 17, 1] and budgeted blocking of identity graphs [2, 5, 6] turn them into fix plans. The algorithmic core of such planning is well studied. What practitioners lack is open evidence about the inputs: how much does a plan depend on identity data, on segmentation, on exploit intelligence, and how does it compare with published planners and with the score-sorted queues that are actually used?

This paper does not propose a new cut algorithm; LINCHPIN’s cut is a vertex max-flow [4]. Its contributions are:

- an open, read-only system that fuses real exports (OpenVAS, Nessus, nmap, SharpHound and a topology overlay) with public exploit intelligence (NVD, EPSS, KEV) into one deterministic

attack graph and returns fix plans in which every fix carries a templated rationale and the paths it breaks (Section 3);

- a seeded benchmark with ground truth and an *ablation* that isolates which data makes plans work, with paired tests and confidence intervals (Sections 4–5);
- comparisons with published planners: the exact budgeted shortest-path interdiction program of Israeli and Wood [9] and a greedy interdiction heuristic in the style of Guo et al. [6];
- a *measured* case study in which continuous integration starts outdated official container images on isolated networks, detects their versions, and checks the resulting plan; and a reproduction of part of Jacobs et al.’s EPSS evaluation [11] with public labels.

2 Related work

Attack graphs and hardening. Phillips and Swiler introduced graph-based vulnerability analysis [15]; Sheyner et al. generated attack graphs by model checking [16]; MulVAL derives them with logic programming [14]. Noel et al. [13] and Wang, Noel and Jajodia [17] compute minimum-cost hardening measures, and Albanese et al. make it time-efficient for large graphs [1]. LINCHPIN’s graph model is deliberately simpler than logical attack graphs: hosts, services, vulnerabilities, privileges, credentials, AD access-control entries and data stores, with an exact vertex cut over remediable nodes.

Identity graphs. Heat-ray combats identity snowball attacks with combinatorial optimization over credential graphs [2]. Guo et al. study budgeted blocking of Active Directory attack graphs with fixed-parameter algorithms [5] and scalable edge blocking with a greedy baseline [6]. LINCHPIN ingests the same kind of SharpHound data but fuses it with vulnerability and segmentation data.

Interdiction. Shortest-path interdiction maximizes the adversary’s shortest path after a budget of removals; Israeli and Wood give a mixed-integer formulation via the dual of the inner shortest-path problem [9]. We use it as the optimal baseline when no cut fits the budget.

Prioritization. EPSS predicts exploitation from public and commercial signals [10]; its third version is evaluated against CVSS and KEV in terms of coverage, efficiency and effort [11]. LINCHPIN uses CVSS, EPSS and KEV only inside its edge costs.

3 System

Findings. Connectors parse exports into one frozen schema, the *normalized finding* (kinds: *cve*, *service*, *credential*, *acl*, *config*, *reachability*), validate it, and merge scanner host identifiers onto topology identifiers through declared aliases. When a scan only reports product versions, an offline index of NVD’s version ranges maps them to CVEs, one finding per service; the fix is then an upgrade. Findings are enriched with the CVSS vector and exploitability sub-score, EPSS and KEV status. LINCHPIN never contacts a host.

Graph. Directed edges follow the attacker: the internet or a host reaches a service (**CAN_REACH**, from segment rules), a service has a vulnerability (**HAS_VULN**), a vulnerability whose CVSS vector implies code execution yields a privilege (**ENABLES**), a privilege controls a host (**LEADS_TO**), a host stores credentials (**STORED_ON**) that grant privileges elsewhere (**GRANTS**), abusable access-control entries grant control of principals and hosts, and a host holds a data store. Each edge gets a deterministic cost in $[0, 1]$:

$$c(e) = \frac{w_1(1 - x_e) + w_2(1 - m_e) + w_3s_e}{w_1 + w_2 + w_3}, \quad (w_1, w_2, w_3) = (0.5, 0.3, 0.2), \quad (1)$$

where x_e is exploitability (0.6 times the normalized CVSS exploitability sub-score plus 0.4 times EPSS, at least 0.95 for KEV entries, and 1 for non-exploit edges), m_e is a prerequisite match (0.5 for credential use whose target is not reachable from where the credential is stored, 1 otherwise), and s_e is a skill penalty per transition class.

Paths, cuts, plans. Paths from entry points to crown jewels are enumerated with Yen’s algorithm [19] on the subgraph that is both reachable from an entry and able to reach a crown jewel. *Chokepoints* come from the dominator tree of a virtual source; the *minimum remediation cut* from a vertex-split max-flow over remediable nodes (vulnerabilities, credentials, access-control entries and non-entry hosts). The plan uses the exact cut whenever it fits the budget, ordered by a greedy set cover of the enumerated paths; otherwise the greedy set cover alone. Every fix carries a rationale from templates (no language model) and the identifiers of the enumerated paths it breaks. A what-if query recomputes paths with nodes removed.

4 Evaluation setup

Topologies. Four families with 50 seeds each and $12 + (\text{seed mod } 49)$ hosts (58 to 317 graph nodes): **single** (one bastion; minimum cut 1), **multi** (parallel bastions and two routes into the database; cut 2, no chokepoint), **none** (flat network; cut about 7, larger than the budget) and **ad** (tiered Active Directory with cached administrator sessions; mean cut 1.34). Every vulnerability uses the CVSS vector, sub-score, EPSS and KEV status of a real CVE from a stratified sample of 3,000 (10% KEV, against about 0.5% in the source population). Every topology also contains two unreachable hosts with KEV-listed critical vulnerabilities and high-CVSS vulnerabilities without code execution: distractors for score-sorted queues.

Planners and metrics. With a budget of three fixes we compare LINCHPIN, its greedy set cover alone, the exact interdiction program [9] solved with HiGHS [8], a greedy interdiction heuristic (repeatedly remove the node on the cheapest path whose removal raises its cost most), CVSS-, EPSS- and KEV-then-EPSS-sorted queues over all vulnerabilities and over vulnerabilities on some entry-to-crown-jewel route only, betweenness centrality, and a random choice. A plan *disconnects* if no crown jewel stays reachable. Where nothing disconnects we report the rise in the attacker’s cheapest-path cost and its share of the optimum. Rates carry 95% Wilson intervals [18], means seeded percentile-bootstrap intervals [3], and paired differences exact McNemar tests [12]. Graph algorithms use NetworkX [7].

Ablation. The same optimizer plans on a degraded view of the findings and the plan is scored on the full graph: without identity findings, with 10/25/50% of them dropped at random, with a flat network, with every CVE treated as code execution, with uniform edge costs, and with identity data only.

planner	disconnected	95% interval	McNemar p vs. LINCHPIN
LINCHPIN (exact cut, else greedy)	150/150	[97.5, 100]	—
Exact interdiction MILP [9]	150/150	[97.5, 100]	1
LINCHPIN greedy set cover only	147/150	[94.3, 99.3]	0.25
Greedy interdiction (Guo et al.-style)	145/150	[92.4, 98.6]	0.063
Betweenness	58/150	[31.2, 46.7]	$< 10^{-4}$
CVSS-first, on-path only	15/150	[6.2, 15.8]	$< 10^{-4}$
KEV then EPSS, on-path only	14/150	[5.6, 15.1]	$< 10^{-4}$
CVSS-first	6/150	[1.8, 8.5]	$< 10^{-4}$
KEV then EPSS	4/150	[1.0, 6.7]	$< 10^{-4}$

Table 1: Disconnection with a budget of three fixes, pooled over the **single**, **multi** and **ad** families (50 seeds each).

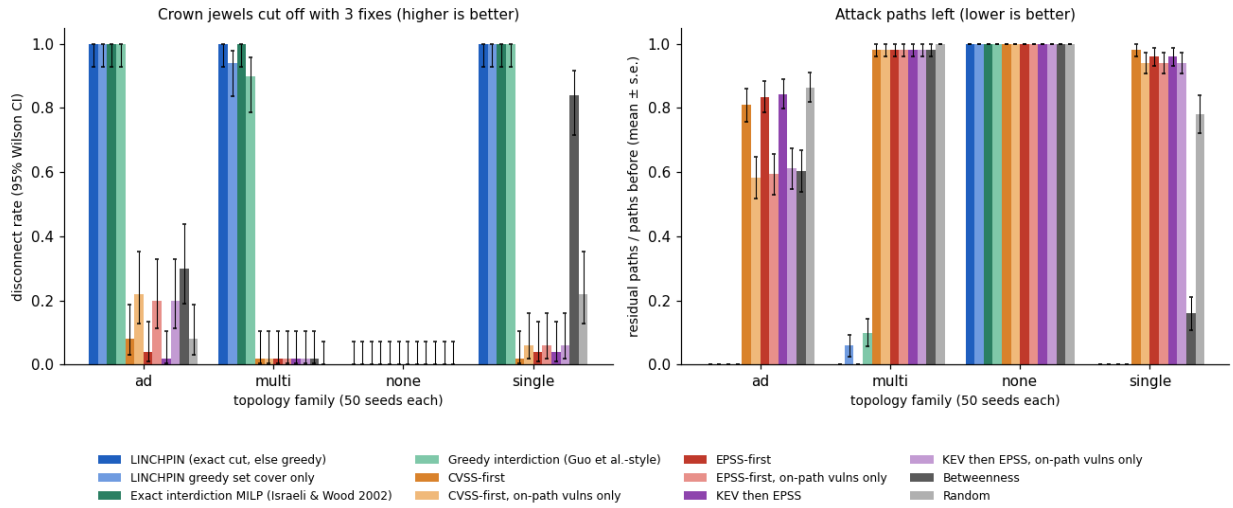


Figure 1: Disconnect rate (left, 95% Wilson intervals) and attack paths left (right, mean $\pm$ s.e.) per family.

What the 100% means. LINCHPIN plans on the graph it is scored on and returns the exact cut whenever that cut fits the budget, so it disconnects every topology where a three-fix cut exists *by construction*. The benchmark measures how far other planners fall short of that optimum inside the model; the ablation measures what incomplete data costs. Neither measures real-world risk.

5 Results

Benchmark. Table 1 pools the 150 topologies of **single**, **multi** and **ad**. Score-sorted queues disconnect 3–4% of them; restricting them to on-path vulnerabilities, which removes the planted distractors, raises this only to 9–10%. Betweenness reaches 39% (84% on **single**, 2% on **multi**). The exact interdiction program also reaches 100%, confirming that the cut itself is not the contribution; the greedy interdiction heuristic misses five **multi** topologies. Across all 200 topologies, on average 69.2% [62.7, 75.5] of the enumerated attack paths are broken by LINCHPIN’s three fixes but not by CVSS-first’s. Figure 1 shows the per-family rates.

planner’s view of the data	disconnected (of 150)	95% interval	McNemar p vs. fused
fused (LINCHPIN)	100%	[97.5, 100]	–
without the exact cut	98.0%	[94.3, 99.3]	0.25
10% / 25% / 50% of identity findings dropped	100 / 95.3 / 90.0%	[84.2, 93.9] (50%)	1 / 0.016 / $< 10^{-4}$
without identity data	36.7%	[29.4, 44.6]	$< 10^{-4}$
identity data only	18.0%	[12.7, 24.9]	$< 10^{-4}$
no graph: KEV then EPSS	2.7%	[1.0, 6.7]	$< 10^{-4}$
flat network / every CVE as RCE / uniform costs	100%	[97.5, 100]	1

Table 2: Ablation: each planner runs the same optimizer on a degraded view and is scored on the full graph.

No cut within budget. On **none** nothing can be disconnected with three fixes. The exact program raises the attacker’s cheapest-path cost by +0.227 [0.209, 0.245] edge-cost units on average; LINCHPIN’s greedy fallback by +0.193 [0.169, 0.216], or 0.83 [0.77, 0.88] of the optimum; greedy interdiction by 0.82; KEV-then-EPSS by 0.15 and CVSS-first by 0.09 of it. Using the exact program as the fallback is an obvious next step.

Ablation. Table 2 shows that identity data is decisive: without it the pooled rate falls to 37%, with a dose-response as more of it is dropped, and identity data alone reaches only 18%. Over-approximating views still disconnect (a cut of a super-graph cuts the real graph) but need more fixes: a flat network 1.82 $\times$ the optimum on **single**, treating every CVE as code execution 1.10 $\times$ on **ad**. The exact cut changes how many fixes are needed more than whether three suffice: greedy alone needs 1.22 $\times$ the optimum on **multi**. Uniform edge costs leave cuts unchanged but reduce the attacker-cost gain on **none** from +0.193 to +0.049, and re-rank paths (Kendall’s τ 0.36–0.51 against the intelligence-based costs).

Measured lab. A continuous-integration job starts official images of httpd 2.4.49, nginx 1.16.1, Tomcat 9.0.30, Redis 5.0.7 and MySQL 5.5.62 on two Docker networks created as internal (no route outside), and runs version detection only (**nmap -sV**, no scripts) from a container inside each network. Segments and firewall rules are derived from what each vantage point saw; that the first network faces the internet and that the MySQL container holds the crown jewel are declared. Five of six services match 393 CVEs by NVD version range (7 in KEV). The minimum cut is one fix; LINCHPIN’s plan (upgrade Tomcat, the only dual-homed container) disconnects the database, as do the interdiction planners; EPSS- and KEV-first need two fixes and CVSS-first three, because its first choice (Redis, CVSS 9.9) lies on only half of the paths.

Reproducing a published comparison. Jacobs et al. report coverage, efficiency and effort of CVSS and EPSS thresholds on CVEs published by the end of 2022, scored on 1 December 2022, against proprietary exploitation telemetry [11]. Restricting to CVEs published by that date with an NVD CVSS v3 score, using the EPSS scores published on that date (EPSS v2) and KEV as the public label, CVSS 7+ flags 58.2% of CVEs (paper: 58.1%) and CVSS 9.1+ 15.0% (15.1%). EPSS needs 40.8% effort to match CVSS 7+’s coverage (paper, EPSS v2: 39.0%; our CVSS 7+ coverage is 89.6% [87.7, 91.3] against the paper’s 82.1%, so this effort is compared at about 90% coverage) and covers 71.4% [68.4, 74.5] at CVSS 9.1+’s effort (paper: 69.9%). The paper’s headline, EPSS v3 at one-eighth of CVSS’s effort, is not reproducible from public data because EPSS v3 scores were not published before March 2023; and because EPSS uses KEV as an input feature, scoring it against

KEV is partly circular. On a prospective label (the 62 CVEs added to KEV in the following year), EPSS v2 still covers 53.2% [41.9, 64.6] at 15% effort against 29.0% [17.7, 41.9] for CVSS 9.1+; with 62 positives these marginal intervals touch, and a paired test on the same positives has not been run. Efficiency does not reproduce (1.6% vs 8.9%) because KEV is a much sparser label than the paper’s exploitation telemetry.

Learned exploitability and scale. An optional logistic-regression model over CVE text and CVSS vectors, trained on CVEs published 2015–2022 with KEV labels known at that date and with sentences that report exploitation status removed, reaches ROC-AUC 0.836 [0.820, 0.852] against 0.754 for the CVSS base score on later CVEs (average precision 0.028 against 0.011). A first version without these leak fixes reported 0.863 and 0.063. Building, ranking and planning take 1.1 s at 467 nodes and 7.0 s at 1,384 nodes on a laptop; Yen’s algorithm dominates. A Neo4j Graph Data Science backend returns identical paths on graphs of up to 238,665 relationships and runs Yen $4\text{--}31\times$ faster, but mirroring the graph into Neo4j costs 8–22 s.

6 Limitations and threats to validity

The topology families were written for this project and plans are scored inside LINCHPIN’s own model, so the benchmark measures distance from an optimum, not risk. The real-export case study (not detailed here) uses public sample reports placed into a declared topology; the measured lab is small and declares its internet exposure and crown jewel. Version banners do not show backported fixes, so matched CVE lists are upper bounds. Code execution is inferred from CVSS vectors, and Active Directory certificate services, shadow credentials, group-policy links and trusts are not modelled. KEV is a small, curated proxy for exploitation in the wild. Residual-path counts are capped at $k = 100$; disconnection and cost gain are reported for that reason. Laptop timings vary by up to $1.6\times$ between sessions.

7 Ethics

LINCHPIN is read-only: it parses files and sends no packets, and its output (which fixes matter most) is the same information a defender needs. The only active probing in this work is version detection of containers that the continuous-integration job itself starts on networks without outside routes; no exploit code was written or run, and no third-party host was contacted. Public sample reports of deliberately vulnerable targets are used as data only.

8 Reproducibility

Code, the CVE pool, the CPE index, the committed scans and every result file are in the public repository (<https://github.com/rakshit-737/linchpin>); the downloader pins public datasets by SHA-256 and NVD’s published hashes; all experiments are seeded and deterministic (path ties do not depend on the interpreter’s hash seed). The documentation lists each command with its runtime (<https://rakshit-737.github.io/linchpin/reproduce/>), and the lab, Neo4j and paper builds run in continuous integration.

Acknowledgements

The software, experiments and this manuscript were developed with substantial assistance from Claude, an AI model by Anthropic, used for code, analysis and writing under the author’s direction; the author is responsible for the content. Data come from NIST’s NVD, FIRST’s EPSS, CISA’s KEV catalog, and the DefectDojo and SpecterOps BloodHound test corpora.

References

- [1] Massimiliano Albanese, Sushil Jajodia, and Steven Noel. Time-efficient and cost-effective network hardening using attack graphs. In *IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, pages 1–12, 2012. <https://doi.org/10.1109/DSN.2012.6263942>.
- [2] John Dunagan, Alice X. Zheng, and Daniel R. Simon. Heat-ray: combating identity snowball attacks using machine learning, combinatorial optimization and attack graphs. In *Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles (SOSP)*, pages 305–320, 2009. <https://doi.org/10.1145/1629575.1629605>.
- [3] Bradley Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1):1–26, 1979. <https://doi.org/10.1214/aos/1176344552>.
- [4] Lester R. Ford and Delbert R. Fulkerson. Maximal flow through a network. *Canadian Journal of Mathematics*, 8:399–404, 1956. <https://doi.org/10.4153/CJM-1956-045-5>.
- [5] Mingyu Guo, Jialiang Li, Aneta Neumann, Frank Neumann, and Hung Nguyen. Practical fixed-parameter algorithms for defending Active Directory style attack graphs. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, pages 9360–9367, 2022. <https://doi.org/10.1609/aaai.v36i9.21167>.
- [6] Mingyu Guo, Max Ward, Aneta Neumann, Frank Neumann, and Hung Nguyen. Scalable edge blocking algorithms for defending Active Directory style attack graphs. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pages 5649–5656, 2023. <https://doi.org/10.1609/aaai.v37i5.25701>.
- [7] Aric A. Hagberg, Daniel A. Schult, and Pieter J. Swart. Exploring network structure, dynamics, and function using NetworkX. In *Proceedings of the 7th Python in Science Conference*, pages 11–15, 2008. <https://doi.org/10.25080/tcwg9851>.
- [8] Qi Huangfu and J. A. Julian Hall. Parallelizing the dual revised simplex method. *Mathematical Programming Computation*, 10(1):119–142, 2018. <https://doi.org/10.1007/s12532-017-0130-5>.
- [9] Eitan Israeli and R. Kevin Wood. Shortest-path network interdiction. *Networks*, 40(2):97–111, 2002. <https://doi.org/10.1002/net.10039>.
- [10] Jay Jacobs, Sasha Romanosky, Benjamin Edwards, Idris Adjerid, and Michael Roytman. Exploit prediction scoring system (EPSS). *Digital Threats: Research and Practice*, 2(3):1–17, 2021. <https://doi.org/10.1145/3436242>.

- [11] Jay Jacobs, Sasha Romanosky, Octavian Suci, Benjamin Edwards, and Armin Sarabi. Enhancing vulnerability prioritization: Data-driven exploit predictions with community-driven insights. In *2023 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*, pages 194–206, 2023. Presented at WEIS 2023; arXiv:2302.14172. <https://doi.org/10.1109/EuroSPW59978.2023.00027>.
- [12] Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947. <https://doi.org/10.1007/BF02295996>.
- [13] Steven Noel, Sushil Jajodia, Brian O’Berry, and Michael Jacobs. Efficient minimum-cost network hardening via exploit dependency graphs. In *19th Annual Computer Security Applications Conference (ACSAC)*, pages 86–95, 2003. <https://doi.org/10.1109/CSAC.2003.1254313>.
- [14] Xinming Ou, Sudhakar Govindavajhala, and Andrew W. Appel. MulVAL: A logic-based network security analyzer. In *Proceedings of the 14th USENIX Security Symposium*, pages 113–128, 2005.
- [15] Cynthia Phillips and Laura Painton Swiler. A graph-based system for network-vulnerability analysis. In *Proceedings of the 1998 Workshop on New Security Paradigms (NSPW)*, pages 71–79, 1998. <https://doi.org/10.1145/310889.310919>.
- [16] Oleg Sheyner, Joshua Haines, Somesh Jha, Richard Lippmann, and Jeannette M. Wing. Automated generation and analysis of attack graphs. In *Proceedings of the 2002 IEEE Symposium on Security and Privacy*, pages 273–284, 2002. <https://doi.org/10.1109/SECPRI.2002.1004377>.
- [17] Lingyu Wang, Steven Noel, and Sushil Jajodia. Minimum-cost network hardening using attack graphs. *Computer Communications*, 29(18):3812–3824, 2006. <https://doi.org/10.1016/j.comcom.2006.06.018>.
- [18] Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927. <https://doi.org/10.1080/01621459.1927.10502953>.
- [19] Jin Y. Yen. Finding the K shortest loopless paths in a network. *Management Science*, 17(11):712–716, 1971. <https://doi.org/10.1287/mnsc.17.11.712>.